Cloud Cost Optimizer

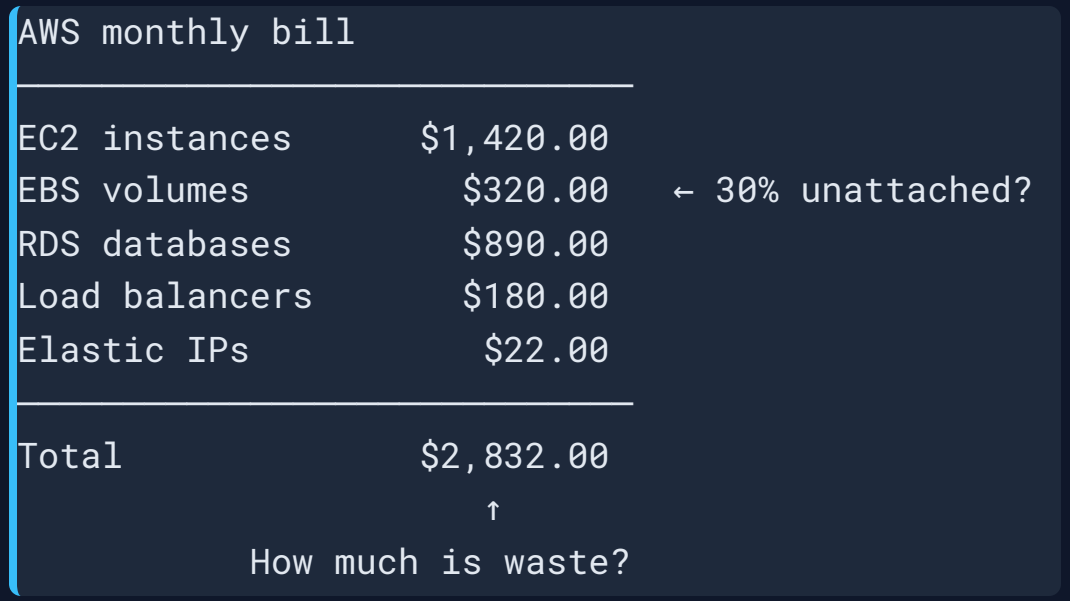
& Remediation Engine

Detect orphaned cloud resources. Generate safe remediation scripts. Ship nothing without review.

Pritam Sonawane · Cloud FinOps Tool · Python / CLI

The Problem: Cloud Waste is Silent

Organizations waste **30–35%** of cloud spend on resources that are running but doing nothing.



Common culprits:

- Unattached disks still charging for storage
- Idle VMs with < 5% CPU — billing hourly

The Core Design Challenge

Billing exports show COST — not STATE

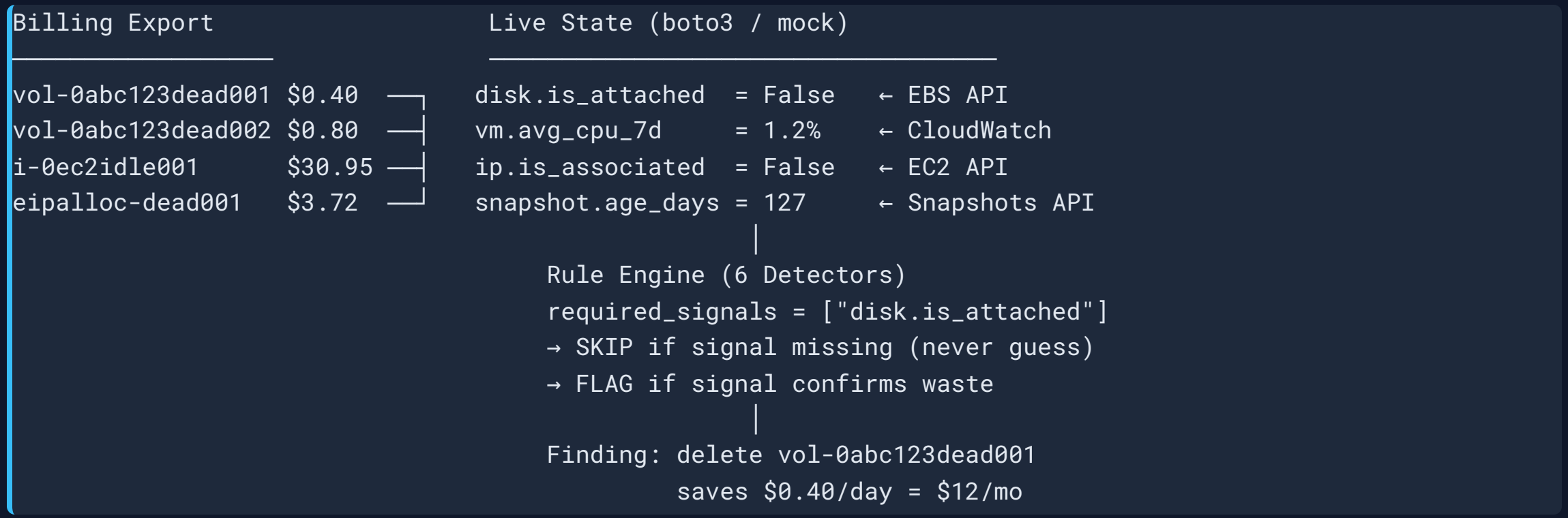
```
AWS Cost & Usage Report (one line per resource per day)

lineItem/ResourceId    lineItem/BlendedCost    lineItem/UsageType
──────────────────────────────────────────────────────────────────
vol-0abc123dead001          $0.40            EBS:VolumeUsage
vol-0abc123dead002          $0.80            EBS:VolumeUsage
```

“ ❓ Is `vol-0abc123dead001` attached to an instance — or floating, billing for nothing? ”

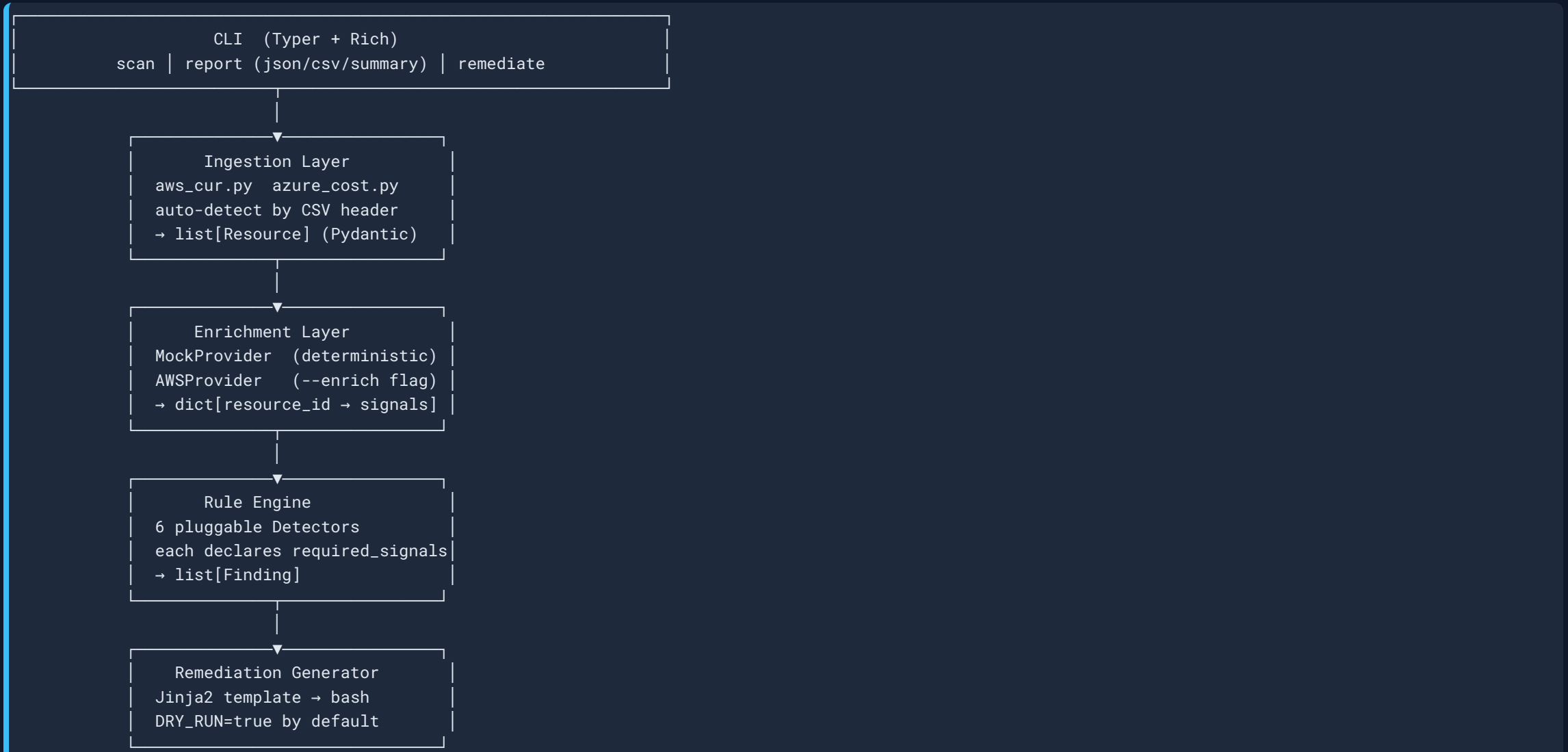
“ **The CUR cannot tell you.** You need a second data source: live

Solution: Enrich Billing Data with Live State



Key invariant: a missing signal means skip, never default to waste.

Architecture



Ingestion & Normalization

Two very different billing formats → **one schema**

AWS CUR (CSV, 30+ columns)

```
# Field mapping
resource_id   ← lineItem/ResourceId
account_id    ← bill/PayerAccountId
region        ← product/region
service       ← lineItem/ProductCode
cost_usd      ← lineItem/BlendedCost
tags          ← resourceTags/user:*
```

Azure Cost Mgmt (CSV + JSON)

```
# Field mapping
resource_id   ← ResourceId (ARM path)
account_id    ← SubscriptionId
region        ← ResourceLocation
service       ← MeterCategory
cost_usd      ← CostInBillingCurrency
tags          ← Tags (JSON string)
```

Unified `Resource` model (Pydantic v2):

```
class Resource(BaseModel):
    resource_id: str;  provider: str;  service: str
```

Detection Engine

Pluggable `BaseDetector` contract

```python
class BaseDetector(ABC):
    name:             ClassVar[str]       # "unattached-disk"
    required_signals: ClassVar[list[str]] # ["disk.is_attached"]

    @abstractmethod
    def detect(self, resources, signals) -> list[Finding]: ...
```

6 Detectors shipped:

Detector	Required Signal	Action
`unattached-disk`	`disk.is_attached`	`aws ec2 delete-volume`
`idle-vm`	`vm.avg_cpu_7d`	`aws ec2 stop-instances`

Remediation & Safety

Three layers of protection before anything runs:

```
# Step 1 — generate the script (no network calls, no state changes)
python -m app.cli remediate billing.csv --output remediation.sh

# Step 2 — read and review the script (it's plain bash, audit it)
cat remediation.sh

# Step 3 — dry-run (default) — prints every command, executes NOTHING
bash remediation.sh
# [DRY-RUN] aws ec2 stop-instances --instance-ids i-0idle00001dead0001 --region us-east-1
# [DRY-RUN] aws ec2 delete-volume --volume-id vol-0orph0001dead0001 --region us-east-1

# Step 4 — execute (requires typed confirmation)
bash remediation.sh --apply
# Type 'yes I understand' to proceed: _
```

Script structure:

Demo — Real Scan on Sample Data

```
$ python -m app.cli scan data/samples/aws_cur_sample.csv
```

```
                        Cloud Cost Findings
 #   Provider  Resource          Type      Category          Savings/mo
 1   AWS       old-etl-…         disk      unattached_disk      $10.00
 2   AWS       stale-ml-…        disk      unattached_disk      $16.00
 3   AWS       old-etl-01        vm        idle_vm              $30.95
 4   AWS       stale-batch-01    vm        idle_vm              $61.90
 5   AWS       old-analytics-…   database  idle_vm              $50.59
 6   AWS       prod-snap-01      snapshot  old_snapshot          $8.00
 7   AWS       orphan-elb-…      lb        idle_load_balancer   $18.00
    ...21 total findings...
┌──────────────────── Savings Summary ────────────────────┐
| Monthly savings : $331.53  |  Annual estimate : $3,978  |
| By Category:  idle_vm $215 · unattached_disk $39 · ...  |
└─────────────────────────────────────────────────────────┘
```

```
$ python -m app.cli scan data/samples/azure_cost_sample.csv
# 11 findings  ·  $142.86/mo  ·  $1,714/yr
```

Tech Stack

Core

- Python 3.13 · Pydantic v2
- Typer (CLI) · Rich (tables/panels)
- Jinja2 (bash script templating)
- PyYAML (rules config)

Testing

- pytest · Typer `CliRunner`

Optional enrichment

- boto3 ≥ 1.34 (`--enrich` flag)
- EC2 `describe-volumes` / `describe-instances`
- CloudWatch `GetMetricStatistics`
- Graceful fallback to mock on credential failure

Challenges & What I Learned

1. Billing ≠ State — and that distinction runs deep

Billing line items are facts about past usage. Live state is a separate API call.
Conflating the two produces false positives at scale. The enrichment signal layer keeps them cleanly separated.

2. Safety-first enrichment

On any API error or empty CloudWatch datapoints, the signal is omitted — never defaulted to a waste-implying value (e.g. `cpu = 0.0` would stop a possibly-healthy instance).
"Omit on error" is a design invariant tested explicitly.

Future Work

Near-term (next sprints)

- **REST API** — FastAPI endpoints: `GET /resources`, `POST /recommendations/{id}/remediate`
- **Persistence** — SQLAlchemy + SQLite: store ingested records, findings, audit log
- **Dashboard** — single-page vanilla-JS UI reading the API

Medium-term

- **Real Azure enrichment** — `az vm list`, `az disk list` via Azure SDK

Thank You

Cloud Cost Optimizer & Remediation Engine

Detect waste · Generate safe remediation · Never delete without review

Resource	Location
Source code	`assignment2/`
Architecture guide	`CLAUDE.md`
Full documentation	`README.md`